



네트워크_03일차

- 1. OSI 7계층에 대해 설명하라
- 2. Restful API란?

▼ 김효선

▼ 1. OSI 7계층에 대해 설명하라

OSI 7계층은 데이터가 송수신 되는 과정을 7개의 계층으로 나눠 정리한 것이다.

송신부는 각 계층마다 필요한 정보를 추가하여 캡슐화를 하여, 수신하게되면 수신부는 역캡슐화를 진행하여 각 계층의 제어 정보를 얻을 수 있다.

물리 계층은 0과 1로 구성된 데이터를 선으로 전달할 수 있도록 전기적인 신호로 변환시켜주는 역할을 한다.

데이터링크 계층은 같은 LAN 안의 연결된 컴퓨터들끼리 MAC주소로 통신이 가능하게 해준다. 이 계층에서 전송되는 단위를 프레임이라고 하고 장비로는 스위치가 있다.

네트워크 계층은 라우터 장비를 이용해 데이터를 목적지까지 안전하고 가장 빠른 경로로 전달해주는 역할을 한다. IP 주소를 이용해 다른 LAN에 속한 컴퓨터로 데이터를 전송할 수 있다.

전송 계층은 양 끝단의 사용자들이 신뢰성 있는 데이터를 주고 받을 수 있도록 해준다. 또한 포트번호로 데이터의 목적지가 어느 애플리케이션인지 식별할 수 있다. 대표적인 프로토콜로는 TCP, UDP가 있다.

세션 계층은 통신하는 두 장치간의 세션(연결)을 관리한다. TCP/IP 세션을 생성하고 유지하며 종료하는 역할을 담당한다.

표현 계층은 데이터 표현에 대한 독립성 제공과 암호화 역할을 담당한다.

응용 계층은 종 사용자와 직접 상호작용하며, 이메일, 웹 브라우저, 파일 전송 등과 같은 응용 서비스를 제공한다.

▼ 2. Restful API란

RESTful API란 **REST** 원칙을 기반으로 설계된 API로, 웹 애플리케이션 간 데이터를 주고받기 위해 주로 사용된다. REST는 자원을 고유하게 식별하고, HTTP URI를 통해 자원을 명시하며, HTTP 메서드를 이용해 해당 자원에 대해 CRUD 작업을 수행하는 방식을 의미한다.

RESTful API는 서버와 클라이언트 간의 통신을 효율적으로 처리할 수 있는 구조로, 주로 **HTTP 프로토콜**을 기반으로 동작한다. 이 API는 간결하고 직관적인 인터페이스를 제공하며, 플랫폼에 종속되지 않아 다양한 클라이언트와 서버 환경에서 사용할 수 있다.

이와 같은 설계는 클라이언트-서버 구조의 웹 기반 애플리케이션에서 자원을 효율적으로 관리하고 상호작용하기에 적합하다.

REST API

- **정의:** REST API는 특정 프레임워크가 아니라 웹 서비스를 설계하는 아키텍처 스타일입니다. REST(Representational State Transfer) 원칙을 따르는 API를 의미합니다.
- **목적:** 클라이언트와 서버 간의 통신을 위한 구조를 제공.
- **특징:**
 - 자원 기반 구조.
 - HTTP 메서드(예: GET, POST, PUT, DELETE)를 사용하여 자원 조작.
 - 상태 무저장(stateless).
 - URI를 통해 자원 식별.

FastAPI

- **정의:** FastAPI는 Python으로 작성된 현대적인 고성능 웹 프레임워크입니다.
- **목적:** 빠르고 쉽게 RESTful API를 만들기 위해 설계됨.
- **특징:**
 - **고성능:** Starlette과 Pydantic을 기반으로 하여 비동기 기능과 데이터 검증을 지원.
 - **자동화된 문서화:** Swagger UI 및 ReDoc을 통해 자동으로 API 문서 생성.
 - **타입 힌트 지원:** Python의 타입 힌트를 사용하여 코드의 가독성과 유지보수성 향상.
 - **비동기 지원:** 비동기 처리 기능을 기본적으로 제공하여 고성능 애플리케이션 구축 가능.

Flask

- **정의:** Flask는 Python으로 작성된 마이크로 웹 프레임워크입니다.
- **목적:** 단순하고 가벼운 웹 애플리케이션을 구축하기 위해 설계됨.
- **특징:**
 - **경량:** 최소한의 구성 요소로 시작하며, 필요에 따라 확장 가능.
 - **유연성:** 플러그인과 확장 기능을 통해 기능 확장 가능.
 - **동기 처리:** 비동기 기능은 기본적으로 지원하지 않으며, 추가 설정이 필요.
 - **쉬운 학습 곡선:** 간단한 구조로 인해 빠르게 학습하고 사용할 수 있음.

비교 요약

특징	REST API	FastAPI	Flask
정의	아키텍처 스타일	고성능 웹 프레임워크	마이크로 웹 프레임워크
주요 목적	클라이언트-서버 통신 구조 제공	빠르고 효율적인 API 구축	단순하고 가벼운 웹 애플리케이션 구축
주요 특징	자원 기반, 상태 무저장	고성능, 비동기 지원, 자동 문서화, 타입 힌트	경량, 유연성, 확장 가능

비동기 지원	기본적으로 지원 안 함	기본적으로 지원	기본적으로 지원 안 함
자동 문서화	없음	Swagger UI, ReDoc 지원	없음
사용 사례	다양한 웹 서비스 및 API 설계	고성능 및 대규모 API	간단한 웹 사이트 및 API